

Ce projet consiste à modéliser un réseau de transports en commun à partir d'un fichier texte décrivant les stations ainsi que leurs différentes liaisons.

Son objectif est d'en permettre une analyse à l'aide de structures de données et d'algorithmes classiques notamment pour le tri, la recherche d'un chemin minimal etc...

Nous avons découpé le code en plusieurs fichiers .c avec leurs fichiers header respectifs:

- station.c qui gère les stations et leur affichage
- edge.c qui s'occupe des liaisons entre les stations via les arcs du graphe (liste d'adjacence)
- hash.c pour la table de hachage servant à identifier les stations par leur nom
- tri.c pour les algorithmes de tri demandés
- dijkstra.c pour l'implémentation de l'algorithme de Dijkstra
- menu.c pour l'interface utilisateur exigée
- et main.c pour le programme principal

Pour les structures de données utilisées :

Nous avons utilisé une première structure STATION (sommet) ainsi que ARC (arête) pour sa liste d'adjacence :

```
typedef struct STATION {  
    int id;  
    char nom[NAME];  
    arc* adj;  
}station;
```

```
typedef struct ARC {  
    int destination;  
    int poids;  
    struct ARC* suivant;  
}arc;
```

Une liste d'adjacence permet d'avoir une représentation plus simple du graphe non orienté et facilite le parcours des voisins effectué plus tard.

La liste d'adjacence est une liste chaînée, ce qui facilite l'insertion itérative des voisins d'une station.

Une table de hachage a aussi été utilisée pour pouvoir faire des recherches sur les stations par leur nom.

```
typedef struct HashNode {  
    char key[NAME];  
    int index;  
    struct HashNode* next;  
}HashNode;
```

```
typedef struct HashTable{  
    HashNode* table[MHASH_SIZE];  
}HashTable;
```

Les collisions dans la table sont gérées par chaînage. Chaque case contient un pointeur vers une liste chaînée de couples (clé (nom), indice), lorsque une collision se produit le nouvel élément est inséré en tête de liste. Lors d'une recherche, la liste est parcourue jusqu'à trouver la clé recherchée. Ceci permet de conserver une complexité quasiment constante tout en s'occupant efficacement des collisions.

Algorithmes :

Tri par sélection : Trouve en boucle le plus petit élément dans la partie non triée puis l'échange avec le premier élément de la partie non triée.

Complexité en temps : $O(n^2)$

Complexité en mémoire : $O(1)$

Tri par insertion : Construit le tableau trié petit à petit en ajoutant à chaque étape le plus petit élément à la bonne place dans le tableau.

Complexité en temps : $O(n^2)$ (worst case) et $O(n)$ (best case)

Complexité en mémoire : $O(1)$

Quick sort : Choisit un pivot dans le tableau et range ce qui est plus petit que le pivot d'un côté et ce qui est plus grand de l'autre puis applique la même chose récursivement sur les tableaux restants

Complexité en temps : $O(n \log n)$

Complexité en mémoire : $O(\log n)$

Nous avons choisi d'implémenter un tri décroissant pour l'option du tri par degré sur le même réseau donné par le sujet, ci-dessous un tableau des valeurs obtenus via notre code :

Algorithme	Comparaisons	Permutations	Déplacements
Tri par sélection	7750	115	X
Tri par insertion	1048	X	1051
Quick sort itératif	5588	257	X
Quick sort récursif	5588	257	X

Le tri par sélection effectue un très grand nombre de comparaisons, ce qui confirme sa complexité théorique en temps en $O(n^2)$. Le nombre de permutations reste cependant limité.

Le tri par insertion présente un nombre de comparaisons bien moins élevé que le tri par sélection qui se justifie par la petite taille du réseau. Il se distingue des autres algorithmes car il ne repose pas sur des permutations d'éléments mais sur des nombreux déplacements successifs, il a donc été plus pertinent de mesurer le nombre de déplacements plutôt que le nombre de permutations.

On retrouve ici une complexité qui se rapproche plus de $O(n)$

Les deux quick sort montrent des résultats peu concluants bien qu'ils paraissent plus performant que le tri par sélection.

Les deux variantes présentent les mêmes résultats, la version itérative évite tout de même l'utilisation de la pile d'appels récursifs.

Pour notre projet le tri par insertion est donc le plus adapté car le réseau que l'on utilise est assez petit.